Bitcoin Illicit Transaction Detection and Fraud Prevention

Team 8

12 December 2025

Contents

1 Introduction **2**

2 Dataset Overview **2**

3 Methodology **2**
3.1 Data Preprocessing . . . 2
3.2 GCN . . . 3
3.2.1 GCN Model Architecture . . . 3
3.2.2 Workflow of GCN Model . . . 3
3.2.3 Rationale for GCN Model . . . 4
3.3 Temporal Splitting Methods . . . 4
3.3.1 Random Split Baseline . . . 5
3.3.2 Chronological Split . . . 5
3.3.3 Rolling Window Split . . . 5
3.4 Model Performance Evaluation . . . 5
3.5 Parameter Tuning . . . 6

4 Experiment Results **6**
4.1 Graph Structure Visualizations . . . 6
4.2 Parameter Tuning and Model Performances . . . 7
4.3 Selected Model . . . 8
4.4 Chronological Patterns . . . 8
4.5 Discussion . . . 9

5 Conclusion **9**

6 References **10**

7 Appendix **11**

1 Introduction

Bitcoin was introduced in 2008 by the pseudonymous person called Satoshi Nakamoto as the first decentralized digital currency (Coinbase, 2025). Instead of being monitored by a central authority, Bitcoin operates on a public ledger known as the blockchain. Every transaction is validated by a global network of nodes, and once recorded, it becomes immutable and timestamped. Although Bitcoin provides convenience and privacy for users, it also creates opportunities for illicit activities such as money laundering and marketplace fraud.

As cryptocurrency becomes popular, it becomes increasingly challenging to correctly identify illicit transactions. There are many tools and models that have been developed. However, traditional machine learning models, such as logistic regression or random forest, can only analyze transactions individually, which means that they cannot capture the relationship between transactions. But in reality, a single transaction may look completely normal when viewed in isolation, but it may be part of a larger fradulent loop.

Graph Convolutional Network (GCN) addresses this problem by incorporating node features and graph structure into the modeling process. GCNs can detect suspicious patterns that may not be obvious if only individual features are analyzed.

In this project, our objective is to classify illicit and licit transactions within the Bitcoin Network using GCN and explore how different data splitting methods affect model performance.

2 Dataset Overview

The Elliptic Dataset is a transaction graph extracted from the Bitcoin network and spans 49 time steps, where each time step corresponds to roughly two weeks of transactions (Clem, 2024). This data set contains 203,769 nodes representing individual transactions and 234,355 directed edges indicating money flow between transactions. Transactions are labeled as licit, illicit, or unknown, and the class distribution is summarized in Table 1.

Label	**Percentage**
Illicit	2.23%
Licit	20.62%
Unknown	77.15%

Table 1: Class Distribution

Each transaction is associated with 166 features. The first 94 features describe local information of a transaction, such as its time step, transaction amount, input/output, etc. The other 72 aggregated features summarize the same local attributes of its neighbors using maximum, minimum, standard deviation, and other aggregate measures.

3 Methodology

3.1 Data Preprocessing

The Elliptic Bitcoin dataset consists of transaction features, directed edges between transactions, and categorical labels indicating whether a transaction is **illicit**, **licit**, or **unknown**. To prepare the data for graph neural network modeling, we applied a multi-step preprocessing pipeline that converts this heterogeneous dataset into a structured graph suitable for PyTorch Geometric (PyG). We first transformed the raw transaction network into a directed graph, where each transaction is represented as a node, and each transfer of funds between transactions generates a directed edge. The edge list was read into memory and converted into the PyG-standard matrix. The dataset uses external transaction identifiers, which we mapped to sequential internal node indices. This mapping allowed us to align nodes with their corresponding 166-dimensional transaction feature vectors, which were stored in the node feature matrix.

Moreover, we label-encoded the Bitcoin transaction classes. The categorical labels were mapped to numerical class indices: 0 = illicit, 1 = licit, and 2 = unknown. Since unknown labels do not contribute a supervised signal, they were excluded from the training and validation masks but remained in the graph to support message passing.

All components, including the feature matrix, edge list, and labels, were merged into a single Data object using PyG's data structure. This unified representation enables batched message passing and efficient GPU computation. Each transaction in the dataset includes a time step between 1 and 49. To help the GCN capture temporal drift in transaction patterns, we normalized the time step values and appended them to the node feature matrix. This results in a final feature dimension of 167 per node.

Finally, we defined Boolean masks (`TRUE` and `FALSE`) indicating which nodes belong to the training, validation, and test sets under the three splitting methods, thereby supporting multiple evaluation strategies. These Boolean masks tell the model which nodes belong to each split, so we can train on one part of the graph and fairly evaluate on another.

Degree centrality measures how many direct connections a node has within a network and is defined as the number of edges connected to that node. Formally,

$$C_D(v) = \frac{\deg(v)}{N-1}$$

where $(\deg(v))$ is the number of neighbors of node (v), and (N) is the total number of nodes in the graph. In this project, we will utilize degree centrality to assess the impact of neighboring transactions' risk on illicit behavior.

3.2 GCN

GCN, or Graph Convolution Network, is from the GNN (Graph Neural Network) family and operates on graph-structured data (Bernstein, 2023), which consists of nodes and edges. Nodes represent entities, and edges capture the relations between entities. Real-world examples of graph-structured data are social networks (node: user, edge: friendship) and transportation networks (node: city, edge: route) (GeeksforGeeks, 2025). A GCN works like a CNN (Convolution Neural Network), but is designed for graphs instead of images, and is widely used in node-level or graph-level classification and link prediction (GeeksforGeeks, 2025).

3.2.1 GCN Model Architecture

The main goal of GCN is to learn and update node embeddings, the vector representations of each node that capture the local feature information. Typically, a GCN consists of one input layer, one or more hidden layers, and one output layer. The input layer takes a set of feature representations of each node. In the hidden layer, the graph convolution operation updates node embeddings by aggregating neighbors' feature information. Then, apply the activation function, such as ReLU, to introduce some non-linearities to the model. The pooling layers are included if we need to reduce the dimensionality of the graph. Finally, the output layer delivers the final node embeddings, which can be used for the downstream tasks such as classification or prediction (GeeksforGeeks, 2025).

3.2.2 Workflow of GCN Model

1. Input Data Preparation. To operate GCN on graph data, it is necessary to represent the graph mathematically. Define a graph G = (V, E), where V refers to the set of nodes and E refers to the set of edges, to represent the nodes and edges in graph-structured data. Each node has its own feature representation matrix X. The adjacency matrix A shows the graph structure by illustrating which nodes are connected. And the degree matrix D is the diagonal matrix that records how many edges are linked to each node (Bernstein, 2023).

2. Graph Preprocessing. Before feeding the data to the convolution layer, there are some preprocessing procedures. First, introduce a self-loop by adding an identity matrix to the adjacency matrix A. This

ensures that each node will not lose its own feature information during future aggregation. Second, the normalized adjacency matrix is computed for maintaining numerical stability. Without normalization, high-degree nodes (nodes with many neighbors) overpower the low-degree nodes (nodes with fewer neighbors) and make the model learn misleading representations. Here is the normalization technique (Bernstein, 2023).

$$\tilde{A} := D^{-\frac{1}{2}}(A+I)D^{-\frac{1}{2}}$$

3. Graph Convolution Layer. This is the main part of the GCN model. In this step, the model updates the vector representation of each node by aggregating its neighbors' feature information. This is the aggregation function that the model used (Kipf, 2016):

$$f(H^{(l)}, A) = \sigma\left(\hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right),$$

where:

- $\hat{A} = A + I$ adjacency matrix with added self-loops
- $\hat{D}$ degree matrix corresponding to $\hat{A}$
- $H^{(l)}$ node features at layer l
- $W^{(l)}$ learnable weight matrix at layer l
- σ activation function (e.g., ReLU)

In a multi-layer GCN, the input to the l-th convolution layer is the updated node representation $H^{(l)}$ produced by the $(l-1)$-th layer. Only the first convolution layer uses the raw node feature matrix X as input.

4. Training Process. In GCNs, the training process is similar to that of other neural networks. The loss function is optimized, usually using optimizers such as SGD and Adam. Some regularization techniques can be used to mitigate overfitting, like dropout (GeeksforGeeks, 2025).

5. Final Output of GCN Model. After training, there are two types of output vectors, depending on the task: node embeddings and graph embeddings. Node embeddings are the input for later downstream node-level tasks such as node classification. In graph embedding, the model aggregates all node embeddings into a single graph-level vector representation through a pooling operation (GeeksforGeeks, 2025). This can be achieved by using pooling functions such as mean pooling or max pooling.

3.2.3 Rationale for GCN Model

A GCN model is an appropriate choice for the Bitcoin dataset application since blockchain transactions are interconnected. It is important that the data is modeled to include these relationships, especially to identify a sequence of illicit transactions. By using a directed graph, the GCN model is able to recognize patterns that appear across multiple transactions, in addition to the transaction-specific information. Traditional machine learning tools cannot capture these details and will likely miss features that can be learned from the blockchain network topology. The GCN model's ability to analyze the graph structure within the Bitcoin dataset will provide strong prediction power to detect illicit transactions. In our GCN model, there are two convolution layers. The loss function is negative log-likelihood loss and the optimizer is Adam.

3.3 Temporal Splitting Methods

Evaluating illicit transaction detection on blockchain data requires careful consideration of how the data is divided into training, validation, and testing subsets. Because illicit behavior exhibits temporal drift, we implement three complementary splitting strategies: a Random Split baseline, a Chronological Split, and a Rolling Window Split. Each method provides a different perspective on the robustness and generalizability of the model under time-evolving transaction patterns.

3.3.1 Random Split Baseline

The random Split serves as a conventional baseline commonly used in machine learning experiments. Under this scheme, all labeled nodes are shuffled uniformly at random, and then assigned to training, validation, and test sets using a specific ratio (80%/10%/10% for train/validate/test split, which is the default for splitting data in open-source implementations). Unknown-labeled nodes remain in the graph for message passing but are excluded from all supervised masks.

This split does not account for temporal ordering; therefore the model may unintentionally train on patterns from future illicit behaviors, which artificially improves test performance. Although unrealistic for fraud detection deployment, the random split establishes an overestimated model accuracy when temporal drift is ignored (Weber et al., 2019).

3.3.2 Chronological Split

To simulate a realistic Anti-money Laundering (AML) deployment setting for fraud-prevention in transactions of Bitcoin, we employ a Chronological Split, where the dataset is partitioned strictly according to transaction time steps, where each time step represents 2 weeks: we trained on earlier time steps (from time step 1 to 29), validated on the middle time steps (from time step 30 to 38), and tested on the most recent time steps (from time step 39 to 49). This setup forces the GCN to learn from past transaction behavior and predict illicit nodes in the future. As a result, the chronological split produces a more challenging assessment of the model's robustness to temporal drift.

3.3.3 Rolling Window Split

The rolling window split evaluates a model that is periodically retrained on recent data, reflecting how financial institutions update fraud detection systems over time. Unlike the chronological split, the rolling window method allows the model to adapt to evolving blockchain behavior by continuously retraining on the most recent data. We divide our trading data chronologically into several window subsets: The step size determines how far the window shifts forward at each iteration, a training window is of fixed length of 24 time steps, a validation window is included within the tail of the training period at the last 12 steps, and a test window immediately following the training window for the next 12 steps. For each window, new label masks are created and a fresh stack GCN is initialized, and training proceeds with early stopping based on validation F1 for the illicit class (see Figure 5 in Appendix). The window is then shifted forward, and the process repeats until the temporal range is exhausted. However, because each window uses a much smaller subgraph with fewer labeled structures and reduced multi-hop[1] connectivity, the rolling-window models often have higher variance and weaker performance compared to the full-graph chronological training.

3.4 Model Performance Evaluation

In this project, we report the F1 score for the illicit class as our primary metric. The F1 score is the harmonic mean of precision and recall, and therefore summarizes in a single value the trade-off between false positives and false negatives, both of which are critical considerations in fraud detection. It is also threshold-based, making it directly interpretable for model selection and comparison across tuning configurations. By contrast, overall accuracy, is not used because the dataset is highly imbalanced, so a model might achieve high accuracy by predicting node as licit while still performing poorly at detecting actual illicit activity.

Although precision-recall curves describe performance across all possible thresholds, the F1 score summarizes performance at a single threshold by balancing precision and recall into one interpretable value. While metrics such as the precision-recall curve (AUC-PR) offer threshold-independent assessments that are useful for exploratory analysis of imbalanced data, they are less convenient for summarizing overall performance in a concise, comparable form. AUC-PR in particular can be sensitive to small fluctuations in predicted probabilities and often varies across temporal splits in a way that does not translate directly into an operational decision threshold. For these reasons, we use the F1 score as the primary performance indicator in

[1] In a graph, a "hop" refers to one step along an edge, so a node's k-hop neighborhood includes all nodes that can be reached by traversing up to k edges from it. The 1-hop neighbors represent direct connections in transaction.

our final results, while PR curves and AUC-PR remain supplemental diagnostics used during exploratory analysis and model-tuning stages.

3.5 Parameter Tuning

To gain a comprehensive understanding of how architecture and training choices can affect GCN's performance on the Elliptic data set, three key parameters are to be tuned. First, stacking K layers collects information from K-hop neighborhoods, while too few layers may underutilize the graph structure and too many layers may lead to over-smoothing, which means node embeddings will become nearly indistinguishable. Second, full-batch training is compared with mini-batch training. With full-batch training, the whole graph is analyzed at once, whereas mini-batch training only samples the neighborhood around the training nodes. Lastly, regularization parameters such as dropout rate and L2 regularization coefficient are tuned to prevent overfitting potentially caused by class imbalance and high dimensionality.

4 Experiment Results

4.1 Graph Structure Visualizations

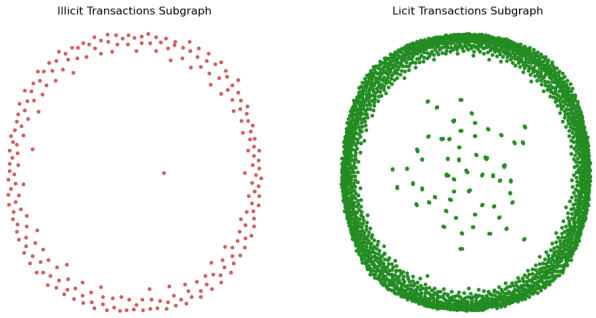

Figure 1: Illicit and Licit Transaction Graphs

The illicit transaction graph (Figure 1) appears relatively sparse, with most nodes positioned around the periphery and only a small number forming a dense central region. This structure suggests that illicit activity is less interconnected overall, with few central nodes potentially acting as hubs that coordinate or route suspicious transactions. In contrast, the licit transaction graph exhibits much higher density and connectivity, indicating a richer and more complex network of interactions. Importantly, the large disparity in the number of licit versus illicit labeled nodes creates a significant class imbalance which we have 20% licit transactions and 2.2% illicit transactions respectively out of all labeled transactions, affecting GCN learning dynamics and motivating the use of negative paired with log-softmax outputs to properly stabilize training under such skewed label distributions.

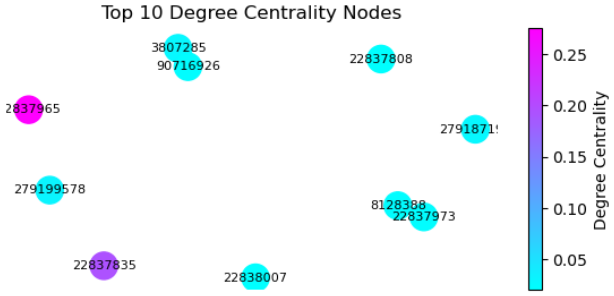

Figure 2: Top 10 Degree Centrality Nodes

In the context of Bitcoin transaction flows, a node with high degree centrality represents a transaction address that interacts with many others, either receiving funds from multiple sources or distributing funds. Figure 2 shows the top 10 nodes ranked by degree centrality in our dataset, the nodes with the darkest color purple is likely to connect with many direct transaction flows. Understanding degree centrality naturally motivates our later analysis of neighborhood illicit risk, because nodes with many neighbors can rapidly transfer illicit funds or interact with known suspicious entities.

4.2 Parameter Tuning and Model Performances

Our baseline setting is shown in Table 2.

Parameters	**Settings**
Number of Layers	2
Training Method	Full-Batch
Dropout Rate	0.5
L2 Regularization	0.0005
Early Stopping	20 Epochs

Table 2: Baseline Parameter Settings

Since the results of parameter tuning are similar across the three data splitting methods and chronological splitting relatively outperforms other methods in terms of the hyper-parameters (see Tables 6-11 in Appendix), only data on the model with chronological split is shown. As we vary the number of GCN layers, we observe that a 2-layer architecture achieves the highest performance (F1 = 0.632). Increasing the depth to three layers leads to a substantial drop in performance (F1 = 0.416), and a 4-layer GCN performs the worst (F1 = 0.046). These results are summarized in Table 3.

Number of Layers	**F1 Score**
2	0.632
3	0.416
4	0.046

Table 3: Layer Experiment (Chronological Split)

Using too many layers causes over-smoothing, a phenomenon where node embeddings become nearly indistinguishable as information is aggregated across increasingly distant neighborhoods. As a result, a deeper GCN fails to separate licit from illicit transactions effectively due to more similar node embeddings.

Then we compared full-batch training and mini-batch training. As shown in Table 4, the 2-layer GCN with full-batch training yields the worst performance (F1 = 0.612), while mini-batch training provides substantial improvements. This performance gap arises because full-batch training unintentionally introduces temporal leakage in chronological data, in which every forward pass exposes the model to future graph structure. In contrast, mini-batch training reduces this problem by only sampling neighborhoods around the training nodes.

Batch Choice	**Sampled Neighborhood**	**F1 Score**
Full-Batch	-	0.612
Mini-Batch	[15,10]	0.751
Mini-Batch	[20,15]	0.744

Table 4: Batch Experiment (Chronological Split)

Finally, we tune regularization parameters to mitigate overfitting. In Table 5, the parameter combination

that yields the best performance (F1 = 0.764) is dropout rate 0.5 and L2 regularization coefficient 0.0005. Too aggressive regularization will distort the learned representations and reduce the model performance.

Dropout Rate	L2 Regularization	F1 Score
0.3	0.0005	0.727
0.5	0.0005	0.764
0.5	0.005	0.708
0.3	0.005	0.670

Table 5: Regularization Experiment (Chronological Split)

4.3 Selected Model

Based on our experiment results, the best model is a 2-layer GCN trained using mini-batch sampling. In this configuration, the first layer samples 15 one-hop neighbors for each training node, and the second layer further samples 10 neighbors from within that previously sampled 15 neighbors. Additionally, applying a dropout rate of 0.5 and L2 regularization coefficient of 0.0005 provides an effective balance between reducing overfitting and maintaining the quality of the learned node representations.

4.4 Chronological Patterns

Figure 3 and Figure 4 reveal how temporal drift and local graph structure shape illicit-transaction detection in our final selected model.

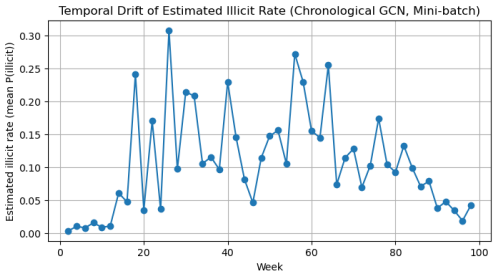

Figure 3: Temporal Drift of Estimated Illicit Rate (Chronological Mini-batch GCN)

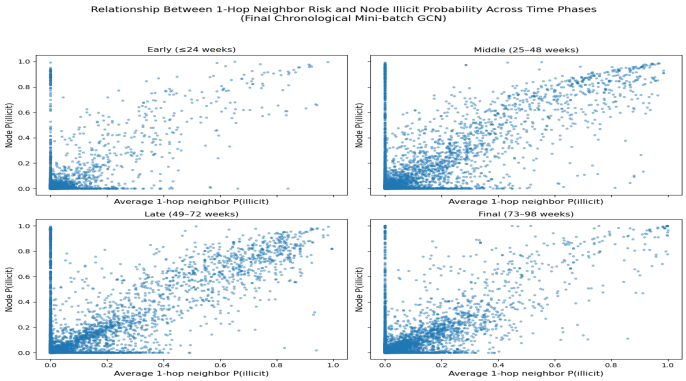

Figure 4: Relationship Between 1-Hop Neighbor Risk and Node Illicit Probability Across Time Phases

Figure 3 plots the estimated illicit rate which is the mean predicted probability of being illicit (y-axis) across weeks (x-axis). The curve shows substantial fluctuation in the early and middle periods (around week 10 to 60), including sharp peaks where the GCN model identifies localized bursts of high-risk activity. In contrast, the estimated illicit rate decreases in the later weeks (after week 70), revealing a temporal drift in blockchain behavior where illicit signals become less concentrated and more difficult for the model to detect.

Figure 4 presents four scatter panels that relate a node's predicted illicit probability (y-axis) to the average illicit probability of its one-hop neighbors (x-axis) across the four temporal phases, where each phase includes 24 natural weeks. There is a strong positive trend in these panels, in particular for the middle period, which implies that nodes surrounded by high-risk neighbors tend to receive higher illicit scores themselves. However, the nodes begin diffusing from the diagonal over time, given more transactions enter the trading market. From this, we can infer that the positive relationship between average direct neighbor illicit risk and estimated illicit rate weakens over the time.

The GCN leverages local structural information, consistent with degree-centrality concept that nodes embedded in high-risk neighborhoods are more suspicious. As illicit nodes become less clustered in later weeks, this neighborhood signal weakens, which aligns with the observed decline in predicted illicit rates and reflects the challenges of modeling evolving blockchain networks.

4.5 Discussion

Even though our model achieved a reasonable F1 score, several limitations remain. First, the extreme class imbalance makes it challenging for our model to learn distinctions between illicit and licit transactions. To overcome this issue, we tried class weighting to artificially increase the importance of the minority illicit class. However, it turned out that the model performance was lower with class weighting, mainly because it destabilized the message passing process and amplified noise. Second, the neighbor sampling used in Mini-Batch GCN may cause the model to miss important patterns, especially when illicit transaction nodes lie out of the sampled subgraph. Third, vanilla GCN is not designed for temporal graphs, which may restrict the model's ability to capture evolving fraudulent patterns within the Bitcoin network.

To further improve model performance, some directions could be taken. First, since vanilla GCN can be viewed as a nontrivial generalization of logistic regression, we may consider combining Random Forest with the GCN output layer to capture both graph structure and complex feature interactions. Second, augmenting node features with temporal encoding may help the model better capture temporal dependencies embedded in the graph. Finally, other models in the GNN family such as GAT and GIN may potentially capture richer relational patterns within the Bitcoin network and they can be implemented to experiment with model improvement.

5 Conclusion

In this project, we introduce GCN architecture and its effectiveness for detecting illicit Bitcoin transactions using The Elliptic Dataset. GCN is able to capture relational patterns that traditional ML models fail to understand. Our experiments show that the 2-layer GCN with mini-batch sampling and chronological data split achieved the best F1 score. Appropriate regularization may further improve the model performance. In general, these results show that GCN is effective in graph-based fraud detection and it has large potential to be extended to other anti-money laundering (AML) practices.

6 References

Bernstein, M. N. (2023, September 24). Graph convolutional neural networks. https://mbernste.github.io/posts/gcn/

Clem, W. (2024) Bitcoin Transactions Graph Neural Networks. Kaggle. https://www.kaggle.com/code/clemwo/bitcoin-transactions-graph-neural-networks/notebook Retrived by December, 2025

GeeksforGeeks. (2025, November 24). Introduction to graph data structure. https://www.geeksforgeeks.org/dsa/introduction to-graphs-data-structure-and-algorithm-tutorials/

GeeksforGeeks. (2025, July 23). Graph convolutional networks (GCNs): Architectural insights and applications. https://www.geeksforgeeks.org/deep-learning/graph-convolutional-networks-gcns-architectural-insights-and-applications/

Kipf, T. (2016, September 30). Graph convolutional networks. https://tkipf.github.io/graph-convolutional-networks/

What is bitcoin? — Coinbase. Coinbase. (2025). https://www.coinbase.com/learn/crypto-basics/what-is-bitcoin

Weber, M. et al. (2019). Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics.Association for Computing Machinery. https://arxiv.org/pdf/1908.02591

Zheng, Z., Zhou, B., & Song, Y . (2025). Temporal-aware graph attention network for cryptocurrency transaction fraud detection. arXiv preprint arXiv:2506.21382. https://arxiv.org/abs/2506.21382

7 Appendix

Number of Layers	F1 Score
2	0.507
3	0.079
4	0.000

Table 6: Layer Experiment (Random Split)

Batch Choice	Sampled Neighborhood	F1 Score
Full-Batch	-	0.548
Mini-Batch	[15,10]	0.783
Mini-Batch	[20,15]	0.762

Table 7: Batch Experiment (Random Split)

Dropout Rate	L2 Regularization	F1 Score
0.3	0.0005	0.793
0.5	0.0005	0.748
0.5	0.005	0.696
0.3	0.005	0.746

Table 8: Regularization Experiment (Random Split)

Number of Layers	F1 Score
2	0.515
3	0.359
4	0.052

Table 9: Layer Experiment (Rolling Window Split)

Batch Choice	Sampled Neighborhood	F1 Score
Full-Batch	-	0.493
Mini-Batch	[15,10]	0.536
Mini-Batch	[20,15]	0.527

Table 10: Batch Experiment (Rolling Window Split)

Dropout Rate	L2 Regularization	F1 Score
0.3	0.0005	0.556
0.5	0.0005	0.501
0.5	0.005	0.544
0.3	0.005	0.491

Table 11: Regularization Experiment (Rolling Window Split)

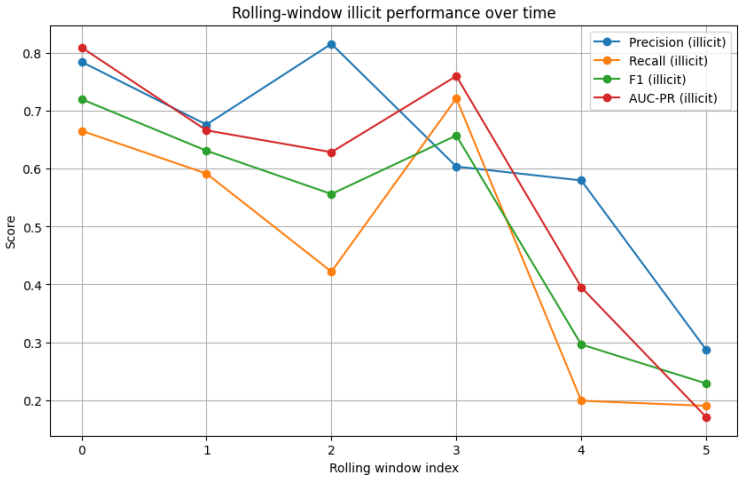

Figure 5: Rolling Window Exploratory Analysis

Figure 5 shows how illicit-class performance changes across sequential time windows when using the untuned GCN model. The downward trend of evaluation metrics reflects temporal drift in the Elliptic dataset: illicit behavior becomes harder to identify as transaction patterns evolve and the illicit class becomes more sparse over time. Because each rolling window trains only on a limited 24-step history, the model has less structural context and fewer labeled examples in later windows, leading to steadily weaker performance.